

## # Code Explanation Document

### ## Project Overview

This project defines a Delta Live Tables (DLT) pipeline on Databricks to ingest CSV customer and order data from Unity Catalog Volumes, cleanse and deduplicate them, compute enriched views (including totals), maintain an SCD Type 2 ordersummary table, and publish an aggregate spend table by customer name and date. It also includes a minimal entrypoint, configuration template for the DLT pipeline, and a basic unit test.

### ## File-by-File Explanation

#### ### dlt\_orders\_pipeline/src/python/main.py

Purpose:

- Minimal CLI entrypoint to validate packaging and provide a user-facing message.

Explanation:

- Defines a main() function that prints a scaffold message and returns via sys.exit when run as a script.

Key Snippet:

```
```python
def main():
    print("dlt_orders_pipeline project scaffold. See README.md for usage.")

if __name__ == "__main__":
    sys.exit(main())
```
```

•

#### ### dlt\_orders\_pipeline/src/python/pipelines/dlt\_orders\_pipeline.py

Purpose:

- Core PySpark + Delta Live Tables (DLT) pipeline script.
- Reads CSV data from Unity Catalog Volumes, cleanses/deduplicates, enriches via joins, performs SCD2 change data capture for ordersummary, and exposes aggregate outputs.

Explanation:

- Constants:
- CATALOG/SCHEMA define target database.

- CUSTOMER\_SOURCE/ORDER\_SOURCE point to UC Volume locations.
- Explicit customer\_schema and order\_schema enforce input structures.
- NORMALIZE\_NULLS lists strings mapped to nulls.
- Source Views:
  - Two DLT views read CSVs with enforced schema and append an ingestion timestamp.
- Cleansing:
  - normalize\_nulls() converts common null-like strings to actual nulls.
  - customer and order tables enforce schema through selects and basic expectations.
- Refinement and Deduplication:
  - customer\_clean: null normalization, drop rows missing CustId, keep latest record per CustId using window order by \_ingest\_ts desc.
  - order\_refined: similar logic for orders; computes TotalAmount; dedupes per OrderId.
  - order\_clean: alias of order\_refined with an expectation on TotalAmount.
- Enrichment:
  - order\_enriched: inner join of customers to orders on CustId with a clean projection.
- SCD2:
  - ordersummary: computes a RecordHash over business attributes; stages rows with SCD2 columns; delegates to dlt.apply\_changes with keys = [CustId, OrderId], sequence\_by = StartDate, stored\_as\_scd\_type=2, and tracking extra columns.
  - ordersummary\_current: seeds an empty table schema as the target for apply\_changes.
- Aggregation:
  - customeraggregatespend: aggregates active SCD2 records (IsActive = true) by Name and Date to produce TotalSpend.

#### Key Snippets:

- Source views with ingest timestamp:

```

```python
@dlt.view(name=f"{SCHEMA}_customer_src")

def customer_src():

    df = spark.read.format("csv").schema(customer_schema).option("header",
    True).load(CUSTOMER_SOURCE)

    df = df.withColumn("_ingest_ts", F.current_timestamp())

    return df

```

```

- Null normalization helper:

```
```python
def normalize_nulls(df: DataFrame) -> DataFrame:
    for c in df.columns:
        if c.startswith("_"):
            continue
        df = df.withColumn(
            c,
            F.when(F.trim(F.col(c)).isin(NORMALIZE_NULLS), F.lit(None)).otherwise(F.col(c))
        )
    return df
```
```

- Deduplication by latest ingestion timestamp:

```
```python
from pyspark.sql.window import Window

w = Window.partitionBy("CustId").orderBy(F.col("_ingest_ts").desc())

df = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
```
```

- Order total and refined projection:

```
```python
df = df.withColumn(
    "TotalAmount",
    (F.col("PricePerUnit") * F.col("Qty")).cast(T.DecimalType(18, 2))
)

return df.select("OrderId", "ItemName", "PricePerUnit", "Qty", "Date", "CustId", "TotalAmount",
    "_ingest_ts")
```
```

- Enrichment join:

```
```python
cust = dlt.read(f'{SCHEMA}_customer_clean').alias("c")
ords = dlt.read(f'{SCHEMA}_order_refined').alias("o")
j = ords.join(cust, F.col("o.CustId") == F.col("c.CustId"), "inner")
```
```

- Record hash for SCD2:

```
```python
src = dlt.read(f'{SCHEMA}_order_enriched').withColumn(
    "RecordHash",
    F.sha2(F.concat_ws(
        "||",
        F.coalesce(F.col("Name"), F.lit("")),
        F.coalesce(F.col("EmailId"), F.lit("")),
        F.coalesce(F.col("Region"), F.lit("")),
        F.coalesce(F.col("ItemName"), F.lit("")),
        F.coalesce(F.col("PricePerUnit").cast("string"), F.lit("")),
        F.coalesce(F.col("Qty").cast("string"), F.lit("")),
        F.coalesce(F.date_format(F.col("Date"), "yyyy-MM-dd"), F.lit("")),
        F.coalesce(F.col("TotalAmount").cast("string"), F.lit(""))
    ), 256)
)
```
```

- Apply SCD2 changes:

```
```python
return dlt.apply_changes(
```

```

target = f"{SCHEMA}_ordersummary_current",

source = staged.alias("s"),

keys = ["CustId", "OrderId"],

sequence_by = "StartDate",

stored_as_scd_type = 2,

track_history_column_list=[

    "Name", "EmailId", "Region", "ItemName", "PricePerUnit", "Qty", "Date", "TotalAmount", "RecordHash", "SurrogateKey"

],

except_column_list=["IsActive", "StartDate", "EndDate"],

)

...

```

- Aggregate spend from active SCD2 rows:

```

```python

cur = dlt.read(f"{SCHEMA}_ordersummary_current").filter(F.col("IsActive") == True)

agg = cur.groupBy("Name",
    "Date").agg(F.sum("TotalAmount").cast(T.DecimalType(18,2)).alias("TotalSpend"))

return agg

...

```

### dlt\_orders\_pipeline/configs/pipeline\_settings.json

Purpose:

- Template configuration for a Databricks DLT pipeline deployment.

Explanation:

- Defines pipeline name, storage path in DBFS, target catalog.schema, dev mode, cluster spec, and the library pointing to the pipeline script path in the workspace.
- Enables `pipelines.enableTrackHistory` for lineage/history tracking.

Key Snippet:

```
```json
```

```
{
```

```
  "name": "dlt_orders_pipeline",
```

```
  "storage": "dbfs:/pipelines/",
```

```
  "target": "gen_ai_poc_databrickscoe.sdlic_wizard",
```

```
  "continuous": false,
```

```
  "development": true,
```

```
  "edition": "ADVANCED",
```

```
  "clusters": [
```

```
    {
```

```
      "label": "default",
```

```
      "num_workers": 2,
```

```
      "spark_version": "13.3.x-scala2.12",
```

```
      "node_type_id": "Standard_D3_v2"
```

```
    }
```

```
  ],
```

```
  "libraries": [
```

```
    {
```

```
      "notebook": {
```

```
        "path": "/Workspace/Repos//dlt_orders_pipeline/src/python/pipelines/dlt_orders_pipeline.py"
```

```
      }
```

```
    }
```

```
  ],
```

```
  "configuration": {
```

```
    "pipelines.enableTrackHistory": "true"
```

```
  }
```

```
}
```

...

### dlt\_orders\_pipeline/tests/test\_main.py

- 

Purpose:

- Minimal unit test to verify the main module imports and exposes main().

Explanation:

- Imports src.python.main via importlib and asserts that main is present.

Key Snippet:

```
```python
```

```
mod = importlib.import_module('src.python.main')
```

```
assert hasattr(mod, 'main')
```

...

### dlt\_orders\_pipeline/README.md

- 

Purpose:

- Project documentation for usage, prerequisites, setup, and deployment.

Explanation:

- Lists repository contents and their roles.
- Specifies environment prerequisites (Databricks with Unity Catalog, DBR version).
- Documents local test execution with pytest.
- Provides step-by-step deployment instructions to create and run a DLT pipeline.
- Notes expectations behavior, SCD2 implementation details, and troubleshooting tips.

Key Snippet:

```
```markdown
```

Deploy to Databricks as a DLT Pipeline

1) Import repo

...

2) Configure pipeline

- Open Workflows > Delta Live Tables > Create Pipeline

...

#### 4) Start the pipeline

- Click Start to run a full refresh.

...